

Tool-Calling LLM Failure Propagation & Scaling Law গবেষণা — সম্পূর্ণ Workflow (০ থেকে শেষ পর্যন্ত)

গবেষণার টাইটেল: *How Tool-Calling Agents Fail as Reasoning Chains Grow: A Large-Scale Taxonomy and Scaling Analysis of Multi-Hop Failure Propagation*

Dataset: ToolBench-v1 (tuandunghcmut/toolbench-v1) — 188K trajectories

এই ডকুমেন্টে পুরো গবেষণা প্রক্রিয়া ০ থেকে শেষ (paper লেখা পর্যন্ত) ধাপে ধাপে বাংলায় ব্যাখ্যা করা হলো, যাতে তুমি একদম বিগিনার হয়েও পুরো কাজটা করতে পারো।

ধাপ ০: প্রস্তুতি (Environment Setup)

০.১ — দরকারি Tools ইনস্টল করো

তুমি Python ব্যবহার করবে। শুরুতে Google Colab ব্যবহার করাই সবচেয়ে সহজ (কোনো ইনস্টলেশন ঝামেলা নেই, ফ্রি GPU পাওয়া যায়)। পরে চাইলে local machine বা university server ব্যবহার করতে পারো।

যা যা লাগবে:

Python 3.10+

Jupyter Notebook / Google Colab

Libraries: `pandas`, `numpy`, `datasets` (HuggingFace), `matplotlib`, `seaborn`, `scikit-learn`, `xgboost`, `shap`, `lifelines` (survival analysis), `mlxtend` (association rule mining), `prefixspan` (sequential pattern mining), `sentence-transformers`, `torch`, `transformers`, `powerlaw` (power-law fitting), `scipy`, `statsmodels`

```
bash
pip install pandas numpy datasets matplotlib seaborn scikit-learn xgboost shap lifelines mlxtend prefixspan sentence-transformers torch transformers powerlaw scipy statsmodels
```

০.২ — Google Colab এ Dataset লোড করো

```
python
from datasets import load_dataset
ds = load_dataset("tuandunghcmut/toolbench-v1", "default")
train = ds["train"]
```

০.৩ — একটা Project Folder Structure বানাও

```
project/
├── data/
│   ├── raw/
│   ├── processed/
│   └── annotated/
├── notebooks/
├── models/
├── results/
│   ├── figures/
│   └── tables/
└── paper/
```

এই structure মেনে চললে পরে paper লেখার সময় সব organized থাকবে — reviewer রা reproducibility পছন্দ করে।

ধাপ ১: Dataset বোঝা ও প্রাথমিক অনুসন্ধান (Data Understanding)

১.১ — Data এর গঠন বোঝা

প্রতিটা row-এ আছে:

`id`: unique identifier, যেটার ভিতরে "Step N:" লেখা থাকে (এই N-ই তোমার **trajectory depth**, যেটা Idea 3-এর জন্য মূল ভ্যারিয়েবল)

`conversations`: dict, যার ভিতরে `from` (system/user/assistant/function) এবং `value` লিস্ট আছে

১.২ — Step Number Extract করে

```
python
import re

def extract_step(id_text):
    match = re.match(r"Step (\d+):", id_text)
    return int(match.group(1)) if match else None

df["step_depth"] = df["id"].apply(extract_step)
```

এই `step_depth` কলামটাই তোমার scaling analysis-এর x-axis হবে।

১.৩ — Exploratory Data Analysis (EDA)

Step depth-এর distribution দেখো (histogram) — কোন depth-এ কত trajectory আছে

`from` sequence-এর length distribution দেখো

কয়টা trajectory-তে `function` turn-এ error/exception শব্দ আছে সেটা খুঁজে বের করো (keyword search: "error", "failed", "invalid", "exception", "timeout", "rate limit")

```
python
error_keywords = ["error", "failed", "invalid", "exception", "timeout", "rate limit"]
def has_error(conv):
    for i, role in enumerate(conv["from"]):
        if role == "function":
            text = conv["value"][i].lower()
            if any(kw in text for kw in error_keywords):
                return True
    return False
df["has_error_turn"] = df["conversations"].apply(has_error)
```

কেন এটা গুরুত্বপূর্ণ: এটাই তোমার প্রাথমিক "failure signal" — কোন trajectory-তে সমস্যা হয়েছে তার প্রথম clue।

ধাপ ২: Data Cleaning ও Preprocessing

২.১ — Duplicate ও Corrupt Records বাদ দাও

```
python
df = df.drop_duplicates(subset="id")
df = df[df["conversations"].notnull()]
```

২.২ — Class Imbalance চেক করো

Dataset card অনুযায়ী step-depth distribution অসম (কম step-এর trajectory বেশি, বেশি step-এর কম)। তাই:

Stratified sampling প্ল্যান করো (প্রতিটা step-depth bucket থেকে proportional sample নেওয়া)

পরে statistical analysis-এ bootstrapped confidence interval ব্যবহার করবে যাতে ছোট sample size-এর bias এড়ানো যায়

২.৩ — Trajectory-কে Structured Format-এ রূপান্তর করো

প্রতিটা trajectory-কে turn-by-turn sequence বানাও:

```
python
```

```
def parse_trajectory(row):
    turns = []
    for role, text in zip(row["conversations"]["from"], row["conversations"]["value"]):
        turns.append({"role": role, "text": text})
    return turns
```

এটা পরের সব ধাপের ভিত্তি (feature engineering, pattern mining, sequence modeling — সবকিছু এই structured format থেকে হবে)।

ধাপ ৩: Feature Engineering

প্রতিটা trajectory থেকে নিচের features বের করো (এগুলো পরে ML মডেল এবং statistical analysis-এ ব্যবহার হবে):

Feature	ব্যাখ্যা
<code>step_depth</code>	মোট কতগুলো step আছে
<code>num_tool_calls</code>	কতবার <code>assistant</code> tool call করেছে
<code>num_distinct_tools</code>	কতগুলো ভিন্ন API/tool ব্যবহার হয়েছে
<code>num_error_turns</code>	কতগুলো <code>function</code> turn-এ error message আছে
<code>query_length</code>	user-এর প্রথম message-এর length (word count)
<code>avg_function_response_length</code>	function response গুলোর গড় length
<code>repeated_tool_call</code>	একই tool বার বার call হয়েছে কিনা (binary)
<code>final_answer_present</code>	trajectory শেষে assistant একটা final answer দিয়েছে কিনা

☐

```
python
def compute_features(turns):
    return {
        "num_tool_calls": sum(1 for t in turns if t["role"] == "assistant" and "\nname\n" in t["text"]),
        "num_error_turns": sum(1 for t in turns if t["role"] == "function" and any(k in t["text"].lower() for k in error_keywords)),
        "query_length": len(turns[1]["text"].split()) if len(turns) > 1 else 0,
        # ... বাকি features এভাবে যোগ করো
    }
```

ধাপ ৪: Failure Label তৈরি (Ground Truth Annotation)

এটা তোমার গবেষণার সবচেয়ে গুরুত্বপূর্ণ ধাপ — কারণ পুরো taxonomy আর scaling analysis এই label-এর উপর নির্ভর করবে।

৪.১ — Sampling Strategy

188K trajectory হাতে annotate করা অসম্ভব। তাই:

প্রতিটা step-depth bucket (Step 2, 3, 4, ... 13+) থেকে **stratified random sample** নাও (যেমন প্রতি bucket থেকে ২০০-৩০০টা করে, মোট ৩,০০০-৫,০০০টা trajectory)

৪.২ — Failure Taxonomy সংজ্ঞায়িত করো (Top-Down)

T-Eval এবং ToolEyes থেকে ধার করে তোমার taxonomy বানাও:

Intent Misunderstanding — user query ভুল বোঝা

Wrong Tool Selection — ভুল API/tool বেছে নেওয়া

Parameter Hallucination — API-কে ভুল/বানানো parameter দেওয়া

Execution/Cascading Error — একটা ভুলের পর একই ভুল বার বার repeat করা বা context হারিয়ে ফেলা

No Failure — সফল trajectory

৪.৩ — LLM-as-Judge দিয়ে Auto-Annotation (Scale করার জন্য)

একটা শক্তিশালী LLM (যেমন GPT-4o বা Claude) কে prompt দিয়ে প্রতিটা sampled trajectory-কে উপরের ৫টা category-র একটাতে label করাও:

System Prompt: তুমি একজন tool-use trajectory evaluator। নিচের trajectory পড়ে বলো এটা কোন failure category-তে পড়ে: [Intent Misunderstanding / Wrong Tool Selection / Parameter Hallucination / Execution Error / No Failure]। শুধু category নাম আর এক লাইন reasoning দাও।

৪.৪ — Human Validation (Gold Subset)

Auto-annotated sample থেকে ~২০০-৩০০টা trajectory তুমি নিজে হাতে label করো (বা ২-৩ জন annotator নিয়ে)। তারপর:

Cohen's Kappa হিসাব করো (LLM label vs human label এর agreement মাপার জন্য)

যদি kappa > 0.6-0.7 হয়, LLM annotation reliable ধরা যায়

```
python
from sklearn.metrics import cohen_kappa_score
kappa = cohen_kappa_score(human_labels, llm_labels)
```

ধাপ ৫: Data Mining দিয়ে Bottom-Up Taxonomy Discovery

এখন top-down taxonomy-কে bottom-up pattern discovery দিয়ে validate/সমৃদ্ধ করবে।

৫.১ — Sequential Pattern Mining

`prefixspan` লাইব্রেরি দিয়ে trajectory-র role-sequence (যেমন: user→assistant→function→assistant→function...) এর মধ্যে কমন failure patterns খুঁজে বের করো:

```
python
from prefixspan import PrefixSpan

sequences = [t["role"] for t in trajectory_turns] # প্রতিটা trajectory এর role sequence
ps = PrefixSpan(all_sequences)
patterns = ps.frequent(minsup=50) # minimum ৫০ বার repeat হওয়া pattern
```

কী খুঁজবে: যেমন `[assistant_call → function_error → assistant_call (same tool)]` — এই প্যাটার্ন যদি বার বার দেখা যায়, এটাই "repeated failure" pattern।

৫.২ — Association Rule Mining

`mlxtend` দিয়ে দেখো কোন feature combination failure-এর সাথে strongly associated:

```
python
from mlxtend.frequent_patterns import apriori, association_rules
# উদাহরণ: {high_step_depth, multiple_tools} → {parameter_hallucination}
```

৫.৩ — Clustering দিয়ে Emergent Failure Types

Trajectory-গুলোকে embed করো (sentence-transformers দিয়ে পুরো conversation-এর embedding নাও), তারপর clustering করো:

```
python
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(trajectory_texts)

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=6, random_state=42)
clusters = kmeans.fit_predict(embeddings)
```

তারপর প্রতিটা cluster-এর trajectory manually দেখে বুঝো এগুলো কোন ধরনের failure represent করে — এবং তোমার top-down taxonomy-র সাথে মিলাও (triangulation)।

ধাপ ৬: Machine Learning — Failure Prediction Model

৬.১ — Target Variable তৈরি

`failure_type` (multi-class: 5 categories) — ধাপ ৪ থেকে পাওয়া label

৬.২ — XGBoost Classifier ট্রেন করো

```
python
import xgboost as xgb
from sklearn.model_selection import train_test_split

X = df[["step_depth", "num_tool_calls", "num_distinct_tools", "num_error_turns", "query_length"]]
y = df["failure_type"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

model = xgb.XGBClassifier(objective="multi:softprob", num_class=5)
model.fit(X_train, y_train)
```

৬.৩ — Evaluation

```
python
from sklearn.metrics import classification_report, confusion_matrix

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

Metrics: Accuracy, Precision, Recall, F1 (per class), Macro-F1

৬.৪ — SHAP দিয়ে Feature Importance Explain করো

```
python
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

কেন গুরুত্বপূর্ণ: এটা তোমাকে বলবে কোন feature (যেমন step_depth) কোন failure type predict করতে সবচেয়ে বেশি সাহায্য করছে — এটাই তোমার "কোন depth-এ কোন failure dominant" claim-এর পরিমাণগত প্রমাণ।

ধাপ ৭: Scaling Law Analysis (Idea 3-এর মূল অংশ)

৭.১ — Failure Rate vs Step Depth Plot করো

```
python
failure_rate_by_depth = df.groupby("step_depth")["is_failure"].mean()
```

Bootstrapped confidence interval যোগ করো (কারণ high-depth bucket-এ sample কম):

```
python
import numpy as np
def bootstrap_ci(data, n_iter=1000):
    means = [np.mean(np.random.choice(data, len(data), replace=True)) for _ in range(n_iter)]
    return np.percentile(means, [2.5, 97.5])
```

৭.২ — Power-Law vs Exponential Fit Test করো

```
python
import powerlaw
fit = powerlaw.Fit(failure_rate_by_depth.values)
R, p = fit.distribution_compare('power_law', 'exponential')
```

AIC/BIC দিয়ে কোন model (power-law, exponential, log-linear) সবচেয়ে ভালো fit করে সেটা যাচাই করো।

৭.৩ — Survival Analysis (Time-to-Failure হিসেবে Depth)

`lifelines` লাইব্রেরি দিয়ে Cox Proportional Hazards model ব্যবহার করো — "trajectory কত step পর্যন্ত টিকে থাকে failure হওয়ার আগে" এটাকে survival problem হিসেবে model করো:

```
python
from lifelines import CoxPHFitter
cph = CoxPHFitter()
cph.fit(df[["step_depth", "num_tool_calls", "is_failure"]], duration_col="step_depth", event_col="is_failure")
cph.print_summary()
```

এটা তোমার সবচেয়ে rigorous statistical contribution — normal scaling curve-এর চেয়ে অনেক বেশি গ্রহণযোগ্য reviewer-দের কাছে।

ধাপ ৮: Deep Learning — Failure Propagation Modeling

৮.১ — Baseline: Markov Chain Transition Model

প্রতিটা step-এ failure-type থেকে পরের step-এর failure-type-এ যাওয়ার probability হিসাব করো:

```
python
import numpy as np
transition_matrix = np.zeros((5, 5)) # 5 failure categories
# প্রতিটা trajectory-র consecutive step pair থেকে count বাড়ানো
```

৮.২ — Advanced: LSTM/Transformer Sequence Model

প্রতিটা turn-কে embed করে (sentence-transformers), একটা sequence model দিয়ে "পরের step-এ failure হবে কিনা" predict করো:

```
python
import torch
import torch.nn as nn

class FailureLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_classes):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])
```

৮.৩ — Markov Chain vs LSTM Comparison

দুইটা মডেলের performance তুলনা করো (accuracy, F1) — এটাই তোমার "simple statistical model vs learned neural model" মেথডোলজিক্যাল contribution।

ধাপ ৯: Cross-Benchmark Validation (Generalizability)

তোমার ToolBench-v1-এ পাওয়া scaling pattern কি অন্য benchmark-এও হয়? এটা validate করতে:

ToolHop (995 queries, multi-hop) অথবা **τ -bench** থেকে ছোট sample নাও
একই feature extraction + failure detection pipeline চালাও
একই scaling curve/survival analysis রিপোর্ট করো
দুই dataset-এর curve মিলে কিনা compare করো (statistical test: KS-test বা curve similarity metric)

ধাপ ১০: Results Synthesis ও Visualization

তৈরি করো:

Taxonomy Table — top-down vs bottom-up (clustering) categories পাশাপাশি
Failure Rate vs Depth curve (with confidence intervals)
SHAP summary plot (feature importance)
Transition matrix heatmap (Markov chain)
Cox model hazard ratio table
Cross-benchmark comparison plot

সব figure `matplotlib`/`seaborn` দিয়ে publication-quality বানাও (high DPI, clear labels, consistent color scheme)।

ধাপ ১১: Paper Writing Structure

Q1 journal-এর জন্য standard structure:

Introduction — motivation, gap, contribution bullet points
Related Work — তোমার ১৬টা paper-এর literature review টেবিল এখানে ব্যবহার হবে
Dataset — ToolBench-v1 বর্ণনা, statistics
Methodology — ধাপ ৪-৯ এখানে বিস্তারিত লিখবে (Data Preparation → Taxonomy → Predictive Modeling → Scaling Analysis → Propagation Modeling → Validation)
Results — সব figure/table সহ
Discussion — কেন এই findings গুরুত্বপূর্ণ, practical implication (যেমন cost-aware verification placement)
Limitations
Conclusion & Future Work

Target Journals: ACM Transactions on Intelligent Systems and Technology (TIST), Expert Systems with Applications, Information Processing & Management, Knowledge-Based Systems, IEEE Access (backup)

সংক্ষিপ্ত Timeline (Suggested)

সপ্তাহ	কাজ
১-২	Environment setup, dataset exploration, feature engineering
৩-৪	Sampling + LLM-based annotation + human validation
৫	Data mining (pattern mining, clustering)
৬-৭	ML model (XGBoost + SHAP)
৮	Scaling law + survival analysis
৯-১০	DL propagation model (Markov + LSTM)
১১	Cross-benchmark validation
১২-১৪	Paper writing, figures, revision

গুরুত্বপূর্ণ পরামর্শ

প্রতিটা ধাপের কোড আর output ভালোভাবে **version control** (Git) এ রাখো

Annotation guideline লিখিতভাবে ডকুমেন্ট করো (reproducibility-এর জন্য পরে দরকার হবে)

প্রতিটা ধাপ শেষে ছোট একটা sanity check করো — যেমন একটা random trajectory manually পড়ে দেখো তোমার feature/label ঠিকমতো বসেছে কিনা

সুপারভাইজার/advisor থাকলে প্রতি ২ সপ্তাহে progress দেখাও, বিশেষ করে annotation আর taxonomy design ধাপে (এখানে subjective judgment লাগে, তাই feedback নেওয়া জরুরি)